

Forge

A proof-producing successor architecture for Lean's grind

Concrete algorithms, integration boundaries,
and executable certificate experiments

September 14, 2026

Abstract

A substantially more capable general-purpose Lean tactic should not replace `grind`'s effective equality and arithmetic machinery with an indiscriminate portfolio. It should combine that forward engine with an explicit planner for proof obligations: backward lemma application, constructive witnesses, generalized induction, and specialized certificates. The key interface is not “try another tactic,” but “request this typed fact, solve its prerequisites, and publish its proof in the correct scope.”

This article specifies such a tactic, provisionally called `forge`. It gives algorithms for demand-directed reasoning, integer-lattice witness extraction, induction-motive generalization and accumulator-invariant synthesis, sparse nonnegative-polynomial certificates, bounded equational exploration, and proof-producing Boolean/theory cooperation. It identifies reusable Lean libraries, separates public tactical composition from version-sensitive internal integration, and specifies a strict trust policy.

Four algorithmic components are implemented in Python. The recorded run contains 833 component test cases, 891 accepted certificate checks, 330 rejected certificate mutations, 150 independent polynomial cross-checks, and 897 recursive-execution cross-checks. The archive includes exact certificates, standard-library-only replay, generated Mathlib proof scripts, and reproducible tests. **Lean itself was not available in the execution environment: the emitted Lean scripts are uncompiled, and this is not an end-to-end benchmark against `grind`.** The experiments establish executable components, not a completed superior Lean tactic.

Reading guide. Sections 1–4 explain the architecture and its soundness boundary. Sections 5–10 specify the main algorithms. Sections 11–14 cover integration, scheduling, and implementation. Sections 15–17 report the experiments and define the benchmark needed to establish superiority. The appendices document the artifact and operational contracts.

Contents

1	The objective: more proof discovery, not more unchecked power	5
1.1	A deliberately strong baseline	5
1.2	What should be delivered first	6
2	A worked cross-component proof	6
3	Architecture: two search levels and an evidence broker	7
3.1	Outer proof planning and inner saturation	7
3.2	The evidence broker	7
3.3	Events instead of repeated blind retries	8
3.4	No forced universal intermediate language	9
4	Soundness, scope, and the precise trust contract	9
4.1	A local evidence invariant	9
4.2	Branch-local conclusions cannot escape	9
4.3	Metavariable ownership and dependent terms	9
4.4	Strict mode and native computation	10
4.5	Failure and countermodels	10
5	Demand-directed theorem application and instantiation	10
5.1	Index conclusions as well as triggers	10
5.2	Constructing missing terms	11
5.3	Tables, cycles, and finite Horn slicing	11
5.4	Relevance without starvation	12
6	Constructive witnesses from integer lattices	12
6.1	The fragment and its advantage	12
6.2	Row-by-row unimodular elimination	13
6.3	A small replay format	13
6.4	Parameterized witnesses and residual inequalities	14
6.5	Cost and implementation boundaries	14
7	Induction selection and invariant synthesis	15
7.1	Choose a motive from recursive call structure	15
7.2	Exact polynomial accumulator templates	15
7.3	Why this template covers polynomial increments	16

7.4	A recovered higher-degree invariant	16
7.5	Beyond arithmetic accumulators	16
8	Ordered nonlinear arithmetic with exact certificates	17
8.1	Add missing positivity structure, not another ring normalizer	17
8.2	Finite dictionary search implemented here	17
8.3	Examples of discovered proof objects	18
8.4	Dictionary growth and a useful limitation	18
8.5	Strictness, rational functions, and discrete types	19
8.6	A bounded analytic extension	19
9	Bounded equational exploration and solver-friendly views	20
9.1	The goal is an advantageous form, not saturation for its own sake	20
9.2	A typed local search algorithm	20
9.3	Dependent rewriting and binder-aware sharing	20
9.4	Sharing with polynomial solvers	21
10	Boolean search, external provers, and reconstruction	21
10.1	A coarse-grained proof-producing SAT/theory loop	21
10.2	Incremental CDCL(T) as a later optimization	22
10.3	Certificate formats and trust profiles	22
10.4	Use existing higher-order and SMT reconstruction work	22
10.5	Models and synthesis candidates	23
11	Integration with Lean 4.34.0	23
11.1	Stage A: ordinary tactical composition	23
11.2	Stage B: a release-specific internal adapter	23
11.3	Proposed module layout	24
11.4	Reification and proof reconstruction	24
11.5	Proof DAGs, declarations, and final auditing	25
12	Scheduling, resource accounting, and predictable behavior	25
12.1	A preservation lane with an honest budget statement	25
12.2	Tasks have structural resource limits	25
12.3	Fairness and duplicate suppression	26
12.4	Deterministic and interactive modes	26
13	Concrete library choices and dependency policy	26

14 Implementation sequence and acceptance gates	27
14.1 Gate 1: proof-bearing wrapper and reproducibility	27
14.2 Gate 2: witnesses, induction, and exact arithmetic certificates	28
14.3 Gate 3: demand indexing and persistent cooperation	28
14.4 Gate 4: SAT and external reconstruction	28
14.5 What not to implement first	28
15 Executed experiments and their interpretation	28
15.1 Environment, counting, and reproducibility	28
15.2 Nonnegative-polynomial certificates	29
15.3 Integer-lattice extraction	30
15.4 Accumulator invariants	30
15.5 Demand slicing with distractors	30
15.6 What the artifact does not demonstrate	31
16 The benchmark required to establish superiority	31
16.1 Baselines and two distinct experimental questions	31
16.2 Corpus strata	32
16.3 Avoid theorem leakage	32
16.4 Metrics beyond a solved percentage	32
16.5 Required ablations	33
16.6 Soundness and replay regression suite	33
17 Assessment and recommended direction	33
A Artifact map and reproduction	34
B Proposed frontend and diagnostic contract	35
C Certificate boundaries at a glance	36

1 The objective: more proof discovery, not more unchecked power

The proposed tactic should solve goals that currently require a user to discover the right intermediate lemma, strengthen an induction statement, construct a witness, or expose an algebraic certificate. It should also handle Boolean-intensive subproblems without forcing all of their branching through the same search loop. Its output remains an ordinary Lean proof, with an explicitly controlled set of axioms.

There are three distinct meanings of “more powerful.” *Logical strength* would mean admitting additional axioms; that is not the objective. *Search coverage* means finding proofs in additional families without manual intervention. *Efficiency* means solving more of a fixed benchmark within equal total resources. This design targets the latter two, but only component behavior has been measured here. A genuine claim of end-to-end improvement requires the experiment in Section 16.

The central recommendation is a **bidirectional, certificate-oriented architecture**: preserve grind as a forward reasoning component; place an AND/OR planner around it; connect components through proof-bearing obligations and facts; and let expensive or external algorithms suggest artifacts that small, auditable replay procedures validate.

1.1 A deliberately strong baseline

The reference manual supplied in the request resolves to Lean 4.34.0 at the research date; the corresponding release was published on September 14, 2026. The source inspection in this article uses that release tag, not an unpinned development branch.[1, 2]

The baseline is already substantial. grind combines congruence closure, E-matching, case analysis, linear and integer arithmetic, and commutative algebra. Its ring solver uses polynomial rewriting and completion inspired by Gröbner-basis computation. The source also contains extensionality, lookahead, model-based theory combination, and non-chronological backtracking. Calling these features new would misidentify the engineering problem.[3, 4, 5, 6]

The manual explicitly distinguishes grind’s intended workload from large combinatorial Boolean searches and points to `bv_decide` for the latter. This suggests a decomposition boundary, not a reason to discard the existing engine.[3]

Area	Already available	Proposed increment
Equality and algebra	Congruence closure, equational reasoning, polynomial completion	Goal-directed alternative forms and algebraic side obligations; retain the existing solvers
Theorem application	E-matching over available terms and selected theorems	Backward retrieval, missing-premise demands, and checked candidate-term construction
Inductive reasoning	Recursors and induction tactics; previously proved inductive lemmas	Automatically choose motives, generalize changing parameters, and synthesize restricted invariant families
Existentials	Lean constructors and arithmetic tools	Explicit parameterized witness extraction, with checked feasibility conditions
Ordered nonlinear goals	Arithmetic tactics and existing sign preprocessing	Bounded searches for missing squares, higher-degree products, and equality-ideal certificates
Boolean search	Existing case analysis and backtracking; a separate SAT tactic	A budgeted SAT/theory lane with scoped explanations and an explicit replay trust mode

This table describes intended operational improvements, not a theorem that existing automation fails on every example in a row. A suitably chosen imported lemma may make an apparently inductive or nonlinear goal immediate for `grind`.

1.2 What should be delivered first

The first implementation should have a narrow, useful core: a backward planner, constructive witness templates, induction generalization, and the exact polynomial-certificate checker. They add new ways to *discover* a proof while leaving arithmetic and congruence closure to established code. Replacing `grind`'s Boolean engine or integrating a full external SMT proof calculus is a later, substantially larger project.

A superficially attractive alternative is a tactic expression such as

```
first | grind | aesop | nlinarith | omega | duper [*]
```

This is a reasonable control experiment, but not the proposed architecture. It does not, by itself, let an invariant synthesizer ask a sign solver for a premise, let that solver ask the witness engine for a bound, and then return the proved facts to the suspended induction branch. The cooperation protocol is the main design work.

2 A worked cross-component proof

Consider an accumulator defined over natural recursion and rational arithmetic:

$$\begin{aligned}\text{acc}(0, a) &= a, \\ \text{acc}(n + 1, a) &= \text{acc}(n, a + n).\end{aligned}$$

The user's goal is a closed-form expression for $\text{acc}(n, 0)$. Straight induction on that statement yields an induction hypothesis about accumulator value 0, while the recursive call uses accumulator value n . The hypothesis is poorly shaped for the call.

Forge proceeds as follows. It recognizes the recursive argument n and the changing nonstructural parameter a . It proposes the stronger statement

$$\forall a \in \mathbb{Q}, \quad \text{acc}(n, a) = F(n, a),$$

then searches a polynomial template affine in a . Equating the base and step cases produces a rational linear system for template coefficients. The smallest successful tried degree gives

$$F(n, a) = a + \frac{n(n-1)}{2}.$$

The induction branch applies its generalized hypothesis at $a + n$. The arithmetic leaf proves

$$a + n + \frac{n(n-1)}{2} = a + \frac{n(n+1)}{2}.$$

A polynomial normalizer handles this leaf; it is not asked to invent the induction hypothesis. Finally, specialization to $a = 0$ closes the original goal.

The same division of labor applies when the result must satisfy an inequality rather than an equality. After proving a functional invariant, an ordered-algebra component may request non-negativity of a bound or produce a sum-of-squares identity. Every request becomes a proof obligation. A discovered invariant is never inserted into the context before its induction proof is complete.

This example is also an implementation test. The supplied prototype takes the recurrence schema and a polynomial increment as input; it synthesizes the invariant from exact coefficient equations and independently checks the universal base and step identities. It does not infer truth from a finite table of values.

3 Architecture: two search levels and an evidence broker

3.1 Outer proof planning and inner saturation

An outer node is a Lean sequent $\Gamma \vdash G$ with an owned metavariable and a saved elaboration state. An AND edge means that all generated subgoals must be solved: constructor introduction, application of a lemma, or induction cases. OR alternatives represent different lemmas, witnesses, induction variables, or normal forms. Each viable sequent may own an inner grind state for forward reasoning.

The layers should not share a single mutable metavariable context indiscriminately. An OR alternative takes a transactional snapshot. An AND expansion records the dependency relations among its child metavariables, because one child can constrain another. A branch is accepted only after all its obligations are solved and its proof term has been replayed in the parent context.

The inner state is used for inexpensive consequences of already available information. The outer planner handles choices that change proof structure. In particular, introducing an existential witness is not modeled as adding an unexplained constant to the inner solver; it is an application of `Exists.intro` whose witness is checked in the right context.

3.2 The evidence broker

A broker owns canonical records for facts and obligations. The following is a proposed interface, not an existing Lean API:

```

Evidence:
  proposition      : Expr
  proof           : Expr
  localContext    : ContextFingerprint
  dependencies     : Array EvidenceId
  branchScope     : ScopeId
  trustProfile    : AxiomProfile

Obligation:
  target          : Expr
  owner           : MVarId
  branchScope     : ScopeId
  availableFacts  : FactView
  priority        : Priority
  requestedBy     : TaskId
  budget          : ResourceBudget

Proposal:
  candidate       : CandidateData
  replay          : ReplayRecipe
  prerequisites    : Array Obligation

enqueueProposal  : Proposal -> SearchM ProposalId
publishEvidence  : CheckedEvidence -> SearchM Unit

```

The important distinction is between a *proposal* and *evidence*. A numeric optimizer can propose coefficients. A model finder can propose an instantiation. A language model can propose a lemma name or a term. None may publish a fact directly. Only a replay layer that produces a well-typed Lean proof may do so.

A fact is accompanied by the actual proposition and proof expression, not a pretty-printed string. Its identity includes the relevant environment, universe levels, typeclass instances, transparency policy, and local-context information. Equal-looking surface expressions can have different types or meanings after elaboration.

3.3 Events instead of repeated blind retries

Each task declares what changes can make it useful: a new fact with a specified head symbol, a merge involving a watched equality class, a tighter numeric bound, a discharged premise, or a newly available witness constructor. A failed task is suspended on those events. It is not rerun merely because some unrelated solver produced another fact.

For example, an interval-based nonlinear task waiting for $x \leq 1$ should wake when that bound is proved. It should not restart after each unrelated list equality. A theorem application waiting for a proof of $S(t)$ wakes when the corresponding obligation closes. Backward requests can themselves cause relevant theorems or algebraic normal forms to be scheduled.

The broker maintains a monotonically increasing revision per scope and per relevant theory view. A task key contains its premise slice and the revisions it actually consulted. The keys are performance aids; soundness comes from replaying the resulting proof, not from trusting the hash.

3.4 No forced universal intermediate language

A single untyped first-order representation would be convenient for external solvers but dangerous as a universal Lean state. Instead, keep Lean expressions as the authoritative representation and let each component maintain a certified projection: a polynomial view, a finite Boolean abstraction, an integer affine system, or a higher-order clause view.

Each projection records a meaning theorem or a reconstruction recipe. Information may be discarded conservatively for search, but any accepted conclusion must be lifted back to the original expressions. For example, replacing a difficult subterm by a fresh arithmetic atom is safe for proving a universal consequence of the abstraction; a satisfying assignment to that atom need not be realizable by the original subterm.

4 Soundness, scope, and the precise trust contract

4.1 A local evidence invariant

Invariant 4.1. *Every published record in scope s denotes a Lean judgment $\Gamma_s \vdash P$. Its proof has no unresolved metavariables, all free variables belong to Γ_s , and its transitive axiom dependencies satisfy the selected trust profile.*

Suppose every expansion of the outer search produces a Lean proof constructor that transforms proofs of its child obligations into a proof of the parent. Suppose every satellite fact satisfies the invariant. Then any completed root node yields a proof of the original goal. This follows by composition of the edge proofs and final type checking. No assumption about the correctness of search heuristics, external arithmetic, or numeric optimization is required for that conclusion.

The invariant is a design requirement for the future Lean implementation. The Python checkers in this archive validate smaller mathematical certificate languages; they have not been proved correct inside Lean. Calling their output “kernel checked” would conflate two different layers.

4.2 Branch-local conclusions cannot escape

If a search branch assumes A and obtains P , it has established $\Gamma, A \vdash P$, not $\Gamma \vdash P$. To reuse the result outside that branch, discharge the assumption and publish $A \rightarrow P$. A contradiction under decisions $A_1, \dots, A_k$ yields a clause

$$\neg A_1 \vee \dots \vee \neg A_k,$$

with a proof in the parent scope. It does not justify retaining all branch equalities in the parent e-graph.

An e-class identifier is therefore meaningful only within a state lineage and revision. A proof node must retain stable expression identities and reasons; it must not depend on a mutable representative number surviving rollback. External processes receive immutable snapshots, and returned expressions are re-elaborated or reconstructed against the snapshot’s context.

4.3 Metavariable ownership and dependent terms

A useful negative example is the goal $\exists x : \alpha, \forall y : \alpha, x = y$. The witness for x cannot be chosen to be the later-bound y . A naive term synthesizer that sees all displayed variables can

accidentally propose precisely that. The existential task's context must exclude y , and replay must reject any escaped local variable.

Similarly, a proof of $i = j$ does not make values of `Vector α i` and `Vector α j` definitionally interchangeable. The planner must use transport or heterogeneous equality where needed. It must not identify them merely because their indices have been merged in an untyped side representation. Typeclass dictionaries and universe instantiations are part of the elaborated expression, not metadata that can always be erased.

Dependencies also affect rollback. If two AND children share a metavariable, they are not independent tasks merely because their visible goals differ. Either solve their dependency strongly connected component transactionally, or coordinate assignments through a single owner. Parallel tasks may propose assignments, but should not race to mutate a shared elaboration state.

4.4 Strict mode and native computation

The strict profile allows only the selected ambient axioms, typically the usual Lean mathematical foundations, plus explicitly permitted user axioms already present in the problem. It rejects `sorryAx`, new ad hoc axioms, and unapproved native-computation axioms. This check is transitive: hiding an axiom inside an auxiliary declaration does not make it acceptable.

This matters for SAT integration. In the current reference, `bv_decide` explicitly adds axioms associated with code-generator trust. The same reference distinguishes proof-producing `cbv/decide_cbv` evaluation from `native_decide`. Thus “the SAT certificate is checked inside Lean” is not, by itself, a sufficient specification of the trust boundary.[8]

A strict SAT lane must reconstruct propositional proof terms or prove the certificate checker's result through an admissible evaluation route. Replacing native evaluation with proof-producing evaluation can be expensive and requires engineering; it is not an already measured, drop-in performance-preserving modification. A separate opt-in native profile may reuse the existing fast path, but the output must report that profile rather than silently weaken the contract.

4.5 Failure and countermodels

Only a completed proof closes a goal. Resource exhaustion, unsupported syntax, missing library adapters, numerical failure, unsuccessful rational reconstruction, and incomplete external proof support all produce UNKNOWN. They are not evidence that the goal is false.

A model is a proposal for a counterexample. To present it as an actual counterexample to a Lean statement, instantiate the original expression and verify the falsifying statement with the appropriate semantics. Countermodels of an abstraction with unconstrained function applications are often spurious. Models can still be valuable for choosing splits, pruning candidate invariants, or proposing witnesses, provided those uses do not bypass replay.

5 Demand-directed theorem application and instantiation

5.1 Index conclusions as well as triggers

Retain the existing E-matching machinery for forward instantiation. Add a backward index over the conclusions of approved theorems. An entry records the theorem constant, universe and type parameters, its conclusion head, a discrimination-tree key, premise shapes, and a cheap estimate of elaboration cost.

Given demand G , retrieve a bounded batch of conclusions that may unify with G . Apply each candidate transactionally, producing subgoals and a proof constructor. A premise that an inner solver can discharge becomes a satellite request; a premise requiring another lemma becomes another outer node. Facts discovered while solving a premise may be published only in the context where their proofs are valid.

This immediately handles a practical gap in forward-only use. Suppose a library theorem concludes $f(t) \leq g(t)$ under premise $S(t)$, but neither that theorem nor $S(t)$ is currently active in forward search. A conclusion index can discover the theorem from the goal, create demand $S(t)$, and schedule the component capable of proving it. This is not a claim that E-matching could never reproduce the same argument with different patterns and annotations. It removes some manual premise and trigger selection.

Aesop already supplies a useful model for indexed rules, normalization, and best-first AND/OR search. Its safe/unsafe distinction must be respected: a transformation that merely happens to make progress is not automatically a safe, non-backtracked rule.[9]

5.2 Constructing missing terms

Some useful instantiations require a term not already in the forward database. Consider a theorem whose conclusion contains $f(t + 1)$ and a goal involving $f(s)$. Over integers, the candidate $t = s - 1$ can be generated by solving the argument equation $t + 1 = s$. Over naturals, the same surface subtraction has different semantics and requires an appropriate guard.

The procedure is bounded and typed:

```
matchDemand(theorem, goal):
  freshen theorem parameters in a child metavariable context
  unify rigid heads and directly matching arguments
  collect remaining argument equations
  dispatch recognized affine equations to witness synthesis
  enumerate small typed terms only for remaining parameters
  apply the theorem with a complete candidate assignment
  retain every generated side condition as an obligation
  accept only after all obligations and the final proof replay
```

This is *candidate generation plus checked application*, not a replacement for Lean’s definitional equality. An arithmetic identity used to match arguments must appear in the proof as rewriting or transport. It cannot be smuggled into the unifier as an unverified definitional equality.

Term generation should be demand-directed. Prefer local variables, constructor applications, extracted affine expressions, existing subterms, and then a small user-configured grammar. Penalize new function heads and increasing expression depth. Schedule typeclass synthesis and inhabitation obligations explicitly rather than assuming an arbitrary type has an element.

5.3 Tables, cycles, and finite Horn slicing

An obligation table prevents repeated exploration of equivalent demands. Its key may use alpha-normalized binders and canonicalized expressions, but must preserve typing and relevant context. Cycles become suspended dependencies, not coinductive successes. From $P \rightarrow Q$ and $Q \rightarrow P$ alone, the engine must not produce P .

The prototype isolates a simpler fragment: a finite collection of *ground definite Horn rules*. It

indexes rules by conclusion, computes the backward dependency closure of the target, and performs ordinary indexed forward chaining on the selected rules. No higher-order matching, Lean elaboration, or quantified E-matching is implemented in this component.

Proposition 5.1. *For a finite ground definite Horn program, restricting forward chaining to the backward conclusion-dependency closure of a target preserves whether that target belongs to the least model.*

Proof. A derivation of the target is a finite tree of rule applications. Starting from its root, every rule used and every premise of that rule is included by the backward closure. Thus every derivation of the target is retained. Conversely, every rule in the restricted program belongs to the original program, so any restricted derivation is valid in the original. Forward fixed-point evaluation computes derivability in both finite programs. Cycles without a supporting fact do not create a derivation. $\square$

This result does not establish completeness for the proposed full Lean procedure. It validates a scheduling principle in a fragment with a clean semantics. The measured cost includes building the conclusion index over all input rules; the prototype does not obtain a sublinear total runtime merely by activating fewer rules.

5.4 Relevance without starvation

Initial candidate scores can combine rigid-head overlap, shared constants, number of unsolved premises, likely arithmetic fragment, and term-growth cost. A deterministic priority rule is preferable to an opaque learned model for the first implementation. Iterative widening should increase retrieval batch size and term depth when earlier batches exhaust.

A fairness lane occasionally runs the original forward process independently of demand scores. This matters because a useful forward lemma may have no obvious syntactic connection to the current goal until another theorem is instantiated. Demand-directed search is a prioritization policy; a permanently closed relevance filter can make the overall search weaker.

Recent work on learned interventions in `grind` already explores failure-triggered interventions in E-matching and lookahead. Forge should treat that as prior work and an optional policy layer, rather than claim that failure-triggered scheduling is itself a new algorithm.[10]

6 Constructive witnesses from integer lattices

6.1 The fragment and its advantage

The first witness extractor handles integer linear equations

$$Ax = b, \quad A \in \mathbb{Z}^{m \times n}, \quad b \in \mathbb{Z}^m.$$

It returns either a particular integer solution and a basis for all homogeneous freedom, or a replayable divisibility obstruction. It does not search for a witness in a bounded numerical box. This is useful when witness coefficients are large or when later inequalities should be expressed in fewer free parameters.

For the equation $6x + 10y = 2$, the extractor returns

$$(x, y) = (2, -1) + t(-5, 3), \quad t \in \mathbb{Z}.$$

The solution is immediately checked by substitution. More importantly, the trace explains why this family contains *all* solutions, which is needed if an obstruction later in the system is to prove impossibility.

This is not a claim that existing integer tactics lack linear decision procedures. The added interface exposes constructive witness families to the outer proof planner and preserves their residual degrees of freedom.

6.2 Row-by-row unimodular elimination

After processing rows $0, \dots, i-1$, maintain

$$\{x \in \mathbb{Z}^n : A_{<i}x = b_{<i}\} = \{x_0 + Bz : z \in \mathbb{Z}^d\}. \quad (\text{L})$$

Initially $x_0 = 0$, $B = I_n$, and $d = n$. For the next row a , compute

$$v = aB, \quad r = b_i - ax_0.$$

Find an integer unimodular matrix U such that

$$vU = (g, 0, \dots, 0), \quad g \geq 0.$$

If $g = 0$, consistency requires $r = 0$; the solution family is unchanged. If $g > 0$, consistency requires $g \mid r$. For $q = r/g$, update

$$x_0 \leftarrow x_0 + (BU)_{:,0}q, \quad B \leftarrow (BU)_{:,1:}, \quad d \leftarrow d - 1.$$

Failure of either consistency condition is a complete obstruction for the current prefix and hence for the whole system.

The matrix U is constructed with extended-gcd transformations on pairs of columns. Given current pair (a, b) and $sa + tb = g$, use

$$\begin{pmatrix} s & -b/g \\ t & a/g \end{pmatrix}.$$

Its determinant is $(sa + tb)/g = 1$, and multiplication sends the pair to $(g, 0)$. A sign flip makes the final first entry nonnegative.

Proposition 6.1. *Each successful row update preserves invariant (L), and a reported divisibility obstruction proves that no integer solution exists.*

Proof. Because U is unimodular, $z = Uw$ is a bijection of $\mathbb{Z}^d$. The new row equation becomes $gw_0 = r$. For $g > 0$, it has an integer solution precisely when $g \mid r$, in which case $w_0 = r/g$ and all other coordinates remain free. Substitution gives the stated update. For $g = 0$, the row is either redundant or contradictory. Induction over processed rows proves both claims. $\square$

6.3 A small replay format

For each row, store the row number, U , g , r , and either the quotient or an obstruction marker. The independent prototype checker recomputes the current row image and residual, checks all entries are integers, verifies $\det U = \pm 1$, and performs the exact state update. It does not call the extended-gcd construction or the solver.

For $[6 \ 10]$, one trace step is

$$U = \begin{pmatrix} 2 & -5 \\ -1 & 3 \end{pmatrix}, \quad [6 \ 10]U = [2 \ 0], \quad \det U = 1.$$

With right-hand side 1 instead of 2, the same reduction establishes the obstruction $2 \nmid 1$.

The determinant checker uses rational elimination. A production Lean checker may instead accept elementary column operations or an explicitly checked integer inverse. The elementary-operation format is attractive because it has short local correctness lemmas and avoids determinant expansion. For a positive existential goal, checking a returned witness directly is cheaper than verifying completeness of the entire lattice; the latter is necessary only when the family or a negative result is used.

6.4 Parameterized witnesses and residual inequalities

For $Ax = b_0 + Cp$, where p are already available integer parameters, an initial implementation can solve the coefficient columns separately. If $Au_0 = b_0$ and $AU = C$, then

$$x = u_0 + Up$$

is a witness family for every p . This sufficient method misses witnesses that require piecewise divisibility conditions or integer division, so it must not report impossibility when the coefficient-wise construction fails.

A stronger symbolic version carries affine residuals through the same row algorithm. It produces obligations $g \mid r(p)$ and uses a checked integer quotient after those obligations are proved. For example, $2x = p$ requires evenness of p ; a universal integer witness does not exist without that restriction. A divisibility hypothesis can discharge the guard, and a reconstructed quotient term supplies the witness.

If the goal also includes $Cx \leq d$, substitute the lattice parameterization to obtain inequalities in the remaining integer parameters. Send that residual problem to the existing integer solver or a witness-capable Presburger component. Merely dropping the inequalities after solving the equalities would be unsound. Natural-number witnesses additionally require nonnegativity; solutions over $\mathbb{Z}$ do not automatically inhabit $\mathbb{N}$.

6.5 Cost and implementation boundaries

There are at most n dimension-reducing steps. Each row reduction uses a sequence of gcd computations and matrix updates. Intermediate coefficient bit lengths, not just matrix dimensions, can dominate cost. The prototype is a straightforward exact implementation, not an optimized Smith/Hermite-normal-form library. Production limits should bound coefficient bit lengths, matrix entries, and certificate operations, and return UNKNOWN when the budget is exhausted.

The prototype accepts nonempty systems with a numeric integer right-hand side, including rows with zero unknowns. Parameterized extraction, residual inequality solving, and Lean proof construction are specified extensions, not features silently included in its test count.

7 Induction selection and invariant synthesis

7.1 Choose a motive from recursive call structure

An induction planner should inspect the recursive functions and inductive predicates appearing in a stalled goal. For each candidate recursor, record structural arguments, recursive call sites, parameters passed unchanged, parameters transformed in recursive calls, and dependencies among local hypotheses and indices.

A useful first heuristic is to generalize changing nonstructural parameters. If a parameter a is replaced by $t(a, n)$ in a recursive call, an induction hypothesis fixed at the original a is often unusable. Revert a together with hypotheses and local declarations that depend on it, then construct the generalized motive in dependency order. The strength of the resulting statement is a search choice, not a logical consequence of the original target; it must be proved.

The planner tries a small ordered family: no extra generalization; changing nonstructural parameters; and their dependency closure. Repeated failure at the same recursive-call mismatch can increase the generalization set. It should not generalize every variable by default, since that can remove useful constraints and create an unnecessarily strong theorem.

Use actual Lean recursors or the function-induction interface to construct the proof. The reference provides `induction` and `fun_induction`; Forge's work is selecting motives and solving their obligations, not inventing a new induction axiom.[8]

7.2 Exact polynomial accumulator templates

The implemented fragment is

$$\text{go}(0, a) = a, \quad \text{go}(n + 1, a) = \text{go}(n, a + r(n)),$$

where $r \in \mathbb{Q}[n]$, $n \in \mathbb{N}$, and $a \in \mathbb{Q}$. Search for

$$F(n, a) = P(n) + aQ(n)$$

of increasing total degree. The two necessary certificate identities are

$$F(0, a) = a, \tag{1}$$

$$F(n + 1, a) = F(n, a + r(n)). \tag{2}$$

They are linear equations in the unknown coefficients of P and Q . Expand them with exact rational polynomial arithmetic, equate every monomial coefficient, and solve the resulting rational linear system. Finite execution samples play no role in acceptance.

The prototype joins the two systems by introducing a formal tag variable t : coefficients of t^0 encode the base equation and those of t^1 encode the step equation. Equating a single polynomial in n, a, t is then equivalent to checking both systems. This is only an implementation convenience; t does not appear in the final invariant.

Proposition 7.1. *If a polynomial F satisfies (1) and (2), then $\text{go}(n, a) = F(n, a)$ for every natural n and rational a .*

Proof. Induct on n with motive $\forall a, \text{go}(n, a) = F(n, a)$. The base case is (1). For the step, apply the induction hypothesis at $a + r(n)$ and then use (2). Specialization at any chosen accumulator gives the desired original statement. $\square$

7.3 Why this template covers polynomial increments

For $F(n, a) = a + P(n)$, the conditions reduce to

$$P(0) = 0, \quad P(n+1) - P(n) = r(n).$$

The forward-difference operator maps polynomials of degree at most $d+1$ with zero constant term bijectively to polynomials of degree at most d over $\mathbb{Q}$. Indeed, the leading term of Δn^{k+1} is $(k+1)n^k$, so its matrix in the monomial basis is triangular with nonzero diagonal. Therefore every polynomial increment of degree d has a unique polynomial sum of degree at most $d+1$ after the initial condition is fixed.

This proves a meaningful fragment-level coverage result: with an adequate degree cap and exact solving, the restricted accumulator synthesizer can find all such polynomial sums. It does *not* make the broader induction planner complete. Exponential accumulators, nonlinear accumulator updates, multiple interacting recursions, and non-polynomial invariants require other templates or lemmas.

A more numerically and symbolically economical basis is $\binom{n}{k}$, since

$$\Delta \binom{n}{k+1} = \binom{n}{k}.$$

A future implementation can compute coefficients in that basis and reconstruct an equivalent rational polynomial or use binomial identities. The included implementation uses ordinary monomials and exact row reduction, which makes its certificate checker smaller.

7.4 A recovered higher-degree invariant

For $r(n) = n^8$, the prototype recovers

$$\begin{aligned} F(n, a) = a + \frac{n^9}{9} - \frac{n^8}{2} + \frac{2n^7}{3} - \frac{7n^5}{15} \\ + \frac{2n^3}{9} - \frac{n}{30}. \end{aligned}$$

The final statement is a rational identity evaluated at natural n ; the proof does not depend on floating-point arithmetic. An integer-valued presentation is possible after clearing denominators, but that is a separate representation decision with corresponding cast and divisibility obligations.

The template ablation without any accumulator term fails to meet the universal base equation $F(0, a) = a$. That is an expected failure of this particular restricted ansatz, not evidence that the specialized theorem at accumulator 0 is unprovable. Likewise, a degree-four cap is too small for increment n^4 .

7.5 Beyond arithmetic accumulators

The same planning pattern applies to lists. Define a structural reverse `rev` and a tail-recursive reverse accumulator `revAcc`. The useful strengthening is

$$\text{revAcc}(xs, a) = \text{rev}(xs) ++ a.$$

The recursive call changes a to $h :: a$. Generalizing a lets the induction hypothesis match that call, and append associativity closes the step. The new capability is selecting the strengthening and its grammar, not discovering associativity from nothing.

A practical non-polynomial template library can include accumulator composition in monoids, map/fold fusion, length/size measures, and paired invariants relating a result to a structural summary. Generate candidates by typed anti-unification of the target and recursive-call equations. Check each candidate by ordinary induction, using `grind` and specialized solvers at the leaves.

For mutually recursive functions or indexed predicates, use their generated recursors and explicitly related motives. The simple “generalize changed arguments” rule is insufficient by itself. A candidate with an invalid index relation is rejected during elaboration or leaves an unsolved proof obligation; it is not repaired by erasing the index.

8 Ordered nonlinear arithmetic with exact certificates

8.1 Add missing positivity structure, not another ring normalizer

A ring normalizer can prove an identity once the right identity is known. It does not by itself prove that an arbitrary polynomial is nonnegative. Mathlib’s `nlinarith` already adds specific nonnegativity facts and products before invoking linear arithmetic. The proposed addition is a bounded, goal-directed search for useful squares and higher-degree combinations that need not occur explicitly in the original syntax.[11, 12]

For an ordered field, normalize the desired comparison to $p \geq 0$, with proved assumptions $g_i \geq 0$ and $h_k = 0$. Seek the identity

$$p = \sum_j c_j q_j^2 \prod_{i \in I_j} g_i + \sum_k r_k h_k, \quad c_j \in \mathbb{Q}_{\geq 0}. \quad (3)$$

All products and identities are interpreted in the original ordered field after justified normalization. Every term in the first sum is nonnegative and every term in the second sum is zero.

Proposition 8.1. *A certificate of the form (3), with exact polynomial identity and valid premise references, proves $p \geq 0$ under the stated assumptions.*

Proof. Squares are nonnegative. Multiplication and addition preserve nonnegativity for the displayed factors and coefficients. Each $r_k h_k$ vanishes. Substitution into the exact identity proves the claim. $\square$

8.2 Finite dictionary search implemented here

Fix total degree limit D and a maximum assumption-product size s . Enumerate square-free subsets I of the nonnegative premises with $|I| \leq s$. For each product g_I , enumerate monomials of degree at most $\lfloor (D - \deg g_I)/2 \rfloor$, and squares of those monomials and their pairwise sums and differences. Deduplicate expanded polynomials. For each equality h_k , add monomial multiples with total degree at most D as unrestricted-sign columns.

Let a_j be the coefficient vector of the j th positive atom and e_k those of the equality-ideal columns. The search problem is

$$\sum_j c_j a_j + \sum_k u_k e_k = \text{coeff}(p), \quad c_j \geq 0,$$

with unrestricted rational u_k . The prototype sends a floating-point relaxation to SciPy’s HiGHS interface to propose a support, then solves the selected coefficient equations again over exact

rationals. It accepts only if all reconstructed positive weights are nonnegative and the complete identity passes the separate exact checker.[13]

The numeric solver is therefore an untrusted search accelerator. Rounding its output to a nearby rational and trusting residuals below a tolerance would be unsound. Even when exact reconstruction succeeds, a negative coefficient in the reconstructed solution causes rejection.

```
searchCone(problem, degree, productLimit):
  positiveAtoms := boundedSquaresTimesPremiseProducts(problem)
  idealColumns  := boundedMonomialMultiples(problem.equalities)
  support      := floatingLP(positiveAtoms, idealColumns, target)
  if support is unavailable: return UNKNOWN
  coefficients := exactRationalSolve(support, target)
  if coefficients are unavailable: return UNKNOWN
  certificate  := assemble(coefficients)
  if exactCheck(problem, certificate): return certificate
  return UNKNOWN
```

This implementation is not a semidefinite-programming solver and is not complete even for all sums of squares. It can also miss a feasible member of its own finite cone if the numeric support proposal or exact support repair is unfortunate. An exact rational LP backend could eliminate that latter incompleteness, but would not make the finite dictionary universal.

8.3 Examples of discovered proof objects

For quadratic dispersion,

$$x^2 + y^2 + z^2 - xy - yz - zx = \frac{1}{2}((x - y)^2 + (x - z)^2 + (y - z)^2).$$

For its quartic analogue,

$$\begin{aligned} & (x^2 + y^2 + z^2)^2 - 3(x^2y^2 + y^2z^2 + z^2x^2) \\ &= \frac{1}{2}((x^2 - y^2)^2 + (x^2 - z^2)^2 + (y^2 - z^2)^2). \end{aligned}$$

For $0 \leq x \leq 1$, a certificate found for the cubic bound is

$$x - x^3 = x^2(1 - x) + x(1 - x).$$

For the equality assumption $xy = 1$,

$$x^2 + y^2 - 2 = (x - y)^2 + 2(xy - 1).$$

Each of these is a small object to replay: prove squares and premise products nonnegative, eliminate equality multiples, and verify one polynomial identity. Some may also be solved by existing tactics; they are certificate-generation demonstrations, not a list of measured grind failures.

8.4 Dictionary growth and a useful limitation

The number of monomials in n variables of degree at most d is $\binom{n+d}{d}$. Pairwise binomial squares make naive candidate construction quadratic in that count, and premise subsets add a further combinatorial factor. Use variable-incidence components, target-support filtering, sparse joins,

and strict size caps. A candidate filter may sacrifice completeness of the dictionary search, so its rejection reason should remain diagnostic rather than semantic.

The very simple nonnegative polynomial $(2x - 3y)^2$ exposes a limitation of a dictionary restricted to m , $m + n$, and $m - n$. The needed square has unequal coefficients. In fact, representing its $-12xy$ cross term by $(x - y)^2$ requires weight 6, already exceeding the target's x^2 coefficient 4; adding other positive dictionary atoms cannot repair that deficit. This case is an expected miss in the supplied tests.

A concrete next extension is exact rational completion of squares for quadratic blocks. For a symmetric rational Gram matrix G , use a fraction-preserving LDL^T decomposition. Nonnegative diagonal entries yield weighted rational linear-form squares. A zero pivot requires all corresponding residual off-diagonal entries to vanish; otherwise this unpivoted PSD test fails and search must try another route. The resulting squares fit the existing checker without extending its trusted certificate language. This extension is specified, not implemented in the current prototype.

8.5 Strictness, rational functions, and discrete types

A weak nonnegativity certificate does not prove a strict inequality. One sufficient route is to certify $p - \varepsilon \geq 0$ for an explicit positive rational ε . Another is to prove every factor of one contributing term strictly positive and every other term nonnegative. A positive factor times a merely nonnegative factor is not necessarily positive.

For rational functions, denominator clearing must produce sign and nonzero obligations. Multiplying by a denominator of unknown sign is not a valid inequality transformation. Multiplying by its square may be useful once nonzeroness is proved, but the reconstruction must still justify the original expression and totalized division semantics.

For integer goals, clear positive rational denominators and replay in an ordered integer or rational interpretation with cast lemmas. For natural goals, truncated subtraction and division cannot be treated as field operations. Finite fields and `BitVec` values do not support the order assumptions of this certificate. The applicability guard must inspect elaborated types and instances, not just the presence of symbols such as `+` and `*`.

8.6 A bounded analytic extension

For a polynomial on a rational box, conversion to a Bernstein basis offers another small certificate: the basis functions are nonnegative and sum to one, so nonnegative coefficients suffice for nonnegativity on the box. Subdivide only when the current coefficients do not establish the required bound. Each subdivision is a proved cover with explicit endpoint conventions.

For transcendental functions, use library-proved monotonicity and interval remainder bounds to reduce a compact-domain obligation to polynomial inequalities. Do not sample a function and treat the samples as a bound. This path requires a domain-specific theorem library and is a proposed later extension, not part of the recorded experiments.

9 Bounded equational exploration and solver-friendly views

9.1 The goal is an advantageous form, not saturation for its own sake

Equational exploration can expose a square, a recursive-call pattern, a cancellation, or a useful theorem trigger. It should operate on a demand slice with a clear extraction objective. For an arithmetic obligation, prefer a form whose nonlinear terms match an available certificate template. For an induction step, prefer a form that exposes the recursive call under the induction hypothesis. For a list goal, prefer forms that match known append or map lemmas.

This is different from enumerating all rewrites of the entire local context. Expression growth can be extreme even when the e-graph shares common structure. The proposal is a bounded guide-term mechanism associated with a particular obligation, using existing Lean equality reasoning wherever possible.

The historical Lean egg tactic and the accompanying equality-saturation work are valuable implementation references. However, its current repository marks the tactic deprecated and recommends `grind`. It should therefore not be selected as a maintained, mandatory dependency for this design. A release-pinned experiment can still compare its techniques, and the underlying Rust egg library is a possible untrusted candidate engine.[15, 16, 17]

9.2 A typed local search algorithm

Start with the target subterms, selected premise subterms, and a small number of explicitly requested guide terms. Partition them by elaborated type and context. Apply only approved rewrite rules, recording theorem instances and substitutions as explanations. For a conditional rewrite $H \rightarrow l = r$, create a demand for H ; do not merge the equality classes until a proof of H is available.

At each fuel boundary, extract several candidate representatives rather than a single syntactically cheapest term. One representative may have the fewest nodes, another the simplest arithmetic degree, and another the best overlap with a desired lemma. The extracted equality must be reconstructed in Lean before the alternative representation is used as a fact.

A practical cost vector is lexicographic: unsupported constructs; unsolved side conditions; maximum polynomial degree or recursive mismatch count; expression size; estimated replay size. Different obligation kinds choose different leading coordinates. Cyclic equivalence classes require a well-founded extraction procedure; a cycle is not a finite Lean term.

9.3 Dependent rewriting and binder-aware sharing

The authoritative proof representation keeps binders, types, universe arguments, and local variables. Sharing an expression below a binder must use a representation that respects alpha-equivalence and binding depth. Reusing a node under a different local context requires an explicit renaming or substitution checked against its dependencies.

A conservative first implementation refuses difficult heterogeneous or dependent rewrites and falls back to Lean's existing rewriting facilities. That restriction reduces coverage, but is preferable to treating heterogeneous equality as ordinary equality or permitting a locally bound term to escape. More capable handling should come from typed explanation reconstruction, not stronger trust in an untyped e-graph.

9.4 Sharing with polynomial solvers

Do not make the e-graph and the ring solver compete to rediscover every commutative polynomial identity. The ring solver supplies canonical polynomial views and their meaning proofs. The equational layer supplies alternative *factored* or *structured* views, especially useful squares and recursive combinators.

For example, an extracted expression $(x - y)^2 + 2(xy - 1)$ can become a positivity certificate candidate for $x^2 + y^2 - 2$. The coefficient checker verifies the identity independently, while the equality premise $xy - 1 = 0$ discharges the residual term. Conversely, a cone certificate can request a particular difference term $x - y$ as a guide for other matching tasks. The shared object is a proved relation between views, not a shared mutable polynomial whose meaning changes during search.

10 Boolean search, external provers, and reconstruction

10.1 A coarse-grained proof-producing SAT/theory loop

The first Boolean extension can be coarse grained. Extract the propositional skeleton of the selected assumptions and negated target. Represent each theory atom by a Boolean identifier with a stable Lean proposition. Introduce definitional variables only with a proved bridge between the original formula and the clause representation.

Then iterate:

```
while budget remains:
  answer := SAT.solve(currentClauses)
  if answer is UNSAT:
    replay propositional certificate and all clause origins
    lift contradiction to the original Lean goal
    return PROVED only if replay and axiom audit succeed
  if answer is not a complete Boolean assignment:
    return UNKNOWN
  open a temporary scope with the assignment's signed atoms
  ask relevant Lean theory engines for a contradiction
  if a contradiction is proved:
    discharge its used decisions into a learned clause
    add the proved clause; close the temporary scope
  else:
    refine the supported abstraction or return UNKNOWN
```

A learned clause may use all decisions in a conflict; minimizing the proof dependency set is an optimization. A solver claiming inconsistency without a reconstructed contradiction does not supply a clause. Likewise, a Boolean model whose theory obligations remain unresolved does not show that the Lean goal is false.

The distinction from existing non-chronological backtracking is important. The proposed lane maintains a SAT-oriented learned-clause database and uses SAT's search machinery for Boolean-intensive slices. It is not claiming that grind currently lacks backtracking or conflict-sensitive control.[6]

10.2 Incremental CDCL(T) as a later optimization

Once the coarse loop is correct, expose incremental theory propagation and conflict explanations. Each trail level gets a scope or rollback marker. When a theory implies literal L from current literals $A_1, \dots, A_k$, return a proof of the clause $\neg A_1 \vee \dots \vee \neg A_k \vee L$. The SAT engine can use that clause for propagation and conflict analysis.

Persistent theory state, undo logs, equality-class revisions, and proof explanations must agree on rollback. A stale explanation referring to a withdrawn equality is a correctness bug even if the Boolean clause looks plausible. Store explanations in terms of stable atoms and proof dependencies, then reconstruct the final clause from those dependencies.

A full incremental integration is a larger project than calling a SAT executable. It should be justified by measurements showing that repeated coarse-grained theory calls dominate runtime. Otherwise, the simpler architecture has a smaller correctness surface and may be sufficient.

10.3 Certificate formats and trust profiles

LRAT-style certificates are appropriate for the propositional part, with a Lean proof or checker theorem for every introduced theory clause. Bit-vector normalization and bit blasting also need meaning theorems linking the bit-level variables to the original operations. Widths must be concrete and preserved; confusing an unsigned comparison with an integer comparison is not a harmless simplification.

Strict mode has two implementation options: reconstruct a propositional proof DAG from the trace, or use a formally justified checker and prove its successful evaluation without adding forbidden native axioms. The first emphasizes sharing and local proof lemmas; the second emphasizes efficient verified evaluation. Neither route allows an external process's success exit code to become a theorem.

The native-trust profile may call the existing `bv_decide` path directly and record its additional dependencies. Both profiles should expose certificate bytes, final proof size, and checking time. A benchmark must not compare strict replay in one tactic with native replay in another while reporting them as identical trust configurations.

10.4 Use existing higher-order and SMT reconstruction work

Duper is a proof-producing saturation prover with explicit premise input and configurable search portfolios. It is a natural terminal solver for a selected logical slice, especially where a clause-based proof is more appropriate than purely local tactic application. Its own documentation emphasizes the value of a small relevant premise set. Forge's retrieval component can supply that set.[18]

Lean-auto provides a translation framework between Lean and automated provers, including higher-order representations and reconstruction through Duper. It is a candidate adapter, not a justification for casually erasing dependent types or assuming every type is inhabited.[19]

QuerySMT's hint-based reconstruction is particularly relevant: an external SMT solver can suggest useful intermediate information that is reconstructed in Lean, rather than requiring either a monolithic replay of every primitive SMT step or a premise-only reconstruction attempt. Lean-SMT provides another directly relevant integration effort. These projects should be evaluated before writing a new all-theories SMT translator.[20, 21]

A concrete order of preference is: existing Lean tactic on the original expressions; restricted,

proved translation with a known replay path; externally proposed hints replayed on the original expressions; and only then a new proof-format adapter for a demonstrably important unsupported fragment. External cvc5 proof output and its available formats provide raw material, but a generic Alethe or LFSC file is not automatically a Lean proof.[22]

10.5 Models and synthesis candidates

A model-producing SMT engine can help propose terms or detect a false candidate invariant. For a synthesized invariant I , a candidate state falsifying its base or step conditions should be checked against those original conditions before being used as a counterexample in a synthesis loop. If the check succeeds, block that candidate or refine its template; otherwise treat the model as spurious.

The simplest useful policy is deterministic, finite-grammar counterexample-guided synthesis. Enumerate typed candidate expressions in increasing cost, use quick model checks to reject bad candidates, and prove survivors in Lean. A learned model may reorder that enumeration but never changes the acceptance condition. The current artifact uses exact linear coefficient synthesis rather than an SMT-driven CEGIS loop.

11 Integration with Lean 4.34.0

11.1 Stage A: ordinary tactical composition

Implement the first version entirely through ordinary Lean goals. The planner snapshots a goal, tries a structural transformation or candidate lemma, obtains explicit subgoals, and uses existing tactics as bounded leaves. A successful certificate replayer creates a local proved lemma; a subsequent grind call receives that lemma explicitly.

This approach can rebuild grind state repeatedly and therefore leave performance on the table. It nevertheless tests the most important semantic boundaries without maintaining a fork of the core engine. The initial objective is a working cooperation protocol with reliable replay, not maximal incremental efficiency.

The baseline run must see the same imports and environment as the enhanced run. Importing a large library can add simplification rules or theorem attributes; success after that change is not necessarily caused by the new search algorithm. Forge should avoid globally activating experimental rules merely by importing its module.

11.2 Stage B: a release-specific internal adapter

The inspected release uses a continuation-passing control interface for grind actions. Its essential type has a current goal and separate continuations for non-applicability and progress. The continuation lets branching code inspect downstream success, prune unnecessary branches, and generate replayable tactics. The default finish loop prioritizes solver actions, instantiation, splitting, and model-based theory combination.[6, 7]

A version-specific adapter can insert a bounded Forge action at selected stall points and use the satellite-solver state facilities for persistent indexes and certificates. The adapter must respect the action contract: returning non-applicable cannot leave logical state changes behind; progress is different from a closed goal; and a successful replay script must still work after unnecessary earlier steps are pruned.

In particular, a plugin should not emit a tactic step that relies on a case split later removed by non-chronological backtracking. Its proof dependencies determine whether the step remains necessary. The release's existing action machinery explicitly addresses this issue; an extension must cooperate with it rather than log every attempted action as part of the final proof.[6]

Keep this adapter behind one narrow module. The design targets 4.34.0, but internal structures are not promised to remain source compatible with subsequent versions. A public-wrapper fallback is important when an internal adapter is unavailable.

11.3 Proposed module layout

```
Forge/
Core/Goal.lean          -- obligation DAG and ownership
Core/Evidence.lean      -- scoped, proof-bearing facts
Core/Scheduler.lean     -- deterministic queues and budgets
Core/Replay.lean        -- proof construction and audit
Planner/Backward.lean   -- premise retrieval and application
Planner/Witness.lean    -- typed terms and affine witnesses
Planner/Induction.lean  -- motive/generalization search
Solvers/Cone.lean       -- nonnegative-polynomial certificates
Solvers/Lattice.lean    -- constructive integer elimination
Solvers/Equational.lean -- bounded alternative term forms
Solvers/Sat.lean        -- Boolean/theory orchestration
Adapters/PublicTactics.lean
Adapters/Grind434.lean  -- all version-sensitive integration
Adapters/Duper.lean
Adapters/QuerySMT.lean
Frontend.lean           -- syntax, diagnostics, replay output
```

This is a production decomposition, not a claim that these Lean modules are supplied as a working implementation. The archive's executable algorithms are in `prototype/`; its generated Lean files are replay candidates for a compatible Mathlib project.

11.4 Reification and proof reconstruction

A polynomial reifier should return an atom array, a sparse rational polynomial, and a theorem relating the original expression to evaluation of that polynomial. Atoms retain their original types and expressions. A cast normalizer may introduce bridge lemmas, but cannot silently identify integers, naturals, and field elements.

The cone replayer can initially produce a sequence of `sq_nonneg`, `mul_nonneg`, and `add_nonneg` applications, equality elimination, and one `ring` proof. This is the approach used by the supplied Lean emitter. A later reflective checker can reduce proof size and avoid repeated normalization, but then its soundness theorem becomes part of the implementation work.

For lattice witnesses, a positive existential result can be reconstructed by repeated `Exists.intro` and direct arithmetic substitution. To exploit a full lattice basis or a negative trace, prove a generic row-update theorem and apply it per trace operation. The logical statements should mention the complete affine family, so the replay does not accidentally infer impossibility from a non-exhaustive set of candidates.

For accumulator invariants, emit the generalized theorem and apply the real recursor. The base and step identities are replayed through polynomial normalization. An execution trace of finitely

many recursive calls is not a substitute for that induction proof.

11.5 Proof DAGs, declarations, and final auditing

Large derived expressions should be shared through local lets or explicit auxiliary lemmas. A proof DAG avoids exponential duplication when several facts rely on the same algebraic identity. Its cache key includes theorem and environment dependencies, not only the text of the goal.

Before accepting the root proof, instantiate all assigned metavariables, reject unresolved ones, validate free-variable scope, and run the normal Lean declaration-checking path. Audit the transitive constants used by the proof against the trust profile. Auxiliary declarations must go through the same discipline; an unchecked helper theorem cannot be used to hide an inadmissible assumption.

The final artifact should include both a compact replay script and structured provenance. A script that says only `forge` reruns the search; it is not a stable proof-discovery result. Prefer explicit lemma names, chosen witnesses, induction variables, and exact certificate data.

12 Scheduling, resource accounting, and predictable behavior

12.1 A preservation lane with an honest budget statement

Let $S(T, b, E)$ be the goals tactic T solves within budget b in environment E , including the chosen trust policy. Forge can reserve an unchanged execution of `grind` with budget b and then run enhancements with additional budget e . In an idealized deterministic resource model, and assuming the reserved run has identical resources, this preserves the baseline’s successful executions:

$$S(\text{grind}, b, E) \subseteq S(\text{forge}, b + e + \text{overhead}, E).$$

This is **not** dominance at equal total budget. Splitting a fixed budget among many solvers can prevent any one of them from finishing. Parallel execution can also change memory availability and wall-clock behavior. The safest preservation mode is a sequential baseline-first cascade with explicit accounting for orchestration overhead.

A separate equal-budget mode is appropriate for competitive benchmarking. It may allocate less to the baseline, and must not advertise the same preservation guarantee. Both modes are useful; conflating them produces misleading performance claims.

12.2 Tasks have structural resource limits

A wall-clock timeout alone is too coarse. Use limits for new terms, theorem instances, recursion/induction depth, candidate witnesses, polynomial degree, monomial count, clause count, coefficient bit length, certificate bytes, and final checking work. When a cap is reached, return a specific UNKNOWN reason and retain only valid completed facts.

Reasonable initial configuration values are engineering hypotheses, not measured optima: backward batches of 32 then 128 lemmas; witness grammar depth two then four; one induction level before trying nested induction; cone degree limits two, four, then six; and separate limits for larger user-enabled searches. The existing eighth-power accumulator experiment uses its own degree cap ten and should not be confused with these proposed general-tactic defaults.

A task’s score should include anticipated replay cost. A search that finds a million-node cer-

tificate is not necessarily better than one that finds a short proof slightly later. One possible heuristic is

$$\text{score}(a) = \frac{\widehat{\text{Pr}}(\text{useful result} \mid a)}{\hat{t}_{\text{search}}(a) + \hat{t}_{\text{replay}}(a) + \hat{t}_{\text{check}}(a) + \varepsilon}.$$

The estimates rank proposals only; they have no role in proof validity. Initially, fixed hand-tuned priorities and conservative size estimates are enough. Learned estimates should be introduced only after held-out ablation testing.

12.3 Fairness and duplicate suppression

Use separate queues for cheap propagation, structural proof steps, arithmetic certificates, and expensive external jobs. Cheap work gets priority, but an age-based mechanism or periodic expensive-task slot prevents starvation. A task is rescheduled only when its relevant input revision changes or an iterative-widening stage explicitly increases its budget.

Hash equivalent candidates before expensive replay, with the full typed context included. Suppress identical theorem instantiations across e-class representative changes only after mapping them to a stable canonical explanation. Failed attempts are cached with their budget and assumptions; failure under a smaller budget or weaker fact view does not justify suppressing a genuinely stronger later attempt.

12.4 Deterministic and interactive modes

Deterministic mode uses stable ordering, seeded heuristics, logical fuel, pinned dependencies, and fixed tie breakers. Wall-clock timing is still measured, but does not choose among successful candidates when a reproducible replay is requested. Interactive mode may use speculative parallel candidates and accept the first admissible proof, while recording the exact successful choice for deterministic replay.

External tasks should run on immutable serialized problems with bounded input/output and explicit subprocess timeouts. Invalid JSON, unknown proof rules, excessive allocation requests, unsupported symbols, or reconstruction failures terminate that proposal, not the surrounding Lean session. The small Python artifact is a research implementation, not a hardened untrusted-certificate service.

13 Concrete library choices and dependency policy

The following choices minimize duplicated reasoning code. They are integration candidates with different roles, not a promise that their current default branches all compile together on Lean 4.34.0.

Library/component	Intended role and boundary
Lean core grind	Equality, existing theorem instantiation, arithmetic, branching, and proof-bearing forward state. Keep it as the baseline and principal leaf engine.[3, 4]
Mathlib arithmetic tactics	Reuse ring, norm_num, linarith/nlinarith, and positivity infrastructure to replay identities and signs. The new search supplies missing certificates rather than replacing these checkers.[11, 12]

Library/component	Intended role and boundary
Aesop	Reuse or adapt its indexed rule/search abstractions for backward planning. A bespoke outer DAG may still be preferable if incremental grind state and cross-solver obligations do not fit the existing rule lifecycle.[9]
Duper	Prove selected higher-order/logical slices from a retrieved premise set. Begin with its ordinary tactic interface and record the successful configuration for replay.[18]
Lean-auto	Investigate its typed translation and reconstruction interfaces before implementing a fresh bridge to external provers.[19]
QuerySMT / Lean-SMT	Evaluate existing SMT hint and proof reconstruction pathways. Use only supported fragments and reject unreconstructed steps.[20, 21]
cvc5	Optional untrusted source of SMT hints, models, synthesis candidates, or proof objects. The exact feature/format and executable version must be recorded with the adapter.[22, 20]
SAT / CaDiCaL pathway	Reuse existing bit-vector/SAT infrastructure in the explicitly native profile; implement an admissible reconstruction/evaluation route for strict mode.[3, 8]
Rust egg	Optional untrusted alternative-form generator for a restricted typed projection. Do not make deprecated Lean egg a new mandatory dependency.[17, 15]
SciPy / HiGHS	Implemented prototype support proposal for sparse cone certificates. Floating-point answers never constitute proofs.[13]
Python Fraction and SymPy	Fraction supports the exact prototype checkers; SymPy independently cross-checks polynomial operations during testing. It is not called by archived-certificate replay.[14]

For a production Lake project, first choose a mutually compatible Lean/Mathlib toolchain and dependency commits, then write the internal adapter for that version. Do not silently upgrade one package to make compilation succeed while retaining benchmark labels from another version. Record the Lake manifest, Lean toolchain, imported modules, active theorem attributes, external executable versions, and trust profile.

A disabled or unavailable optional component must leave the basic tactic usable. For example, failure to locate an SMT executable should not break local induction or cone-certificate replay. The artifact intentionally does not provide a speculative Lake manifest that pretends untested dependency combinations are known to work.

14 Implementation sequence and acceptance gates

14.1 Gate 1: proof-bearing wrapper and reproducibility

Build the public tactic wrapper, owned goal DAG, resource accounting, saved-state transactions, and root axiom audit. Add the baseline-first mode and explicit replay output. The gate is passed when successful baseline examples remain successful with identical imports, failed speculative branches leave no state behind, and replay closes every generated proof without rerunning discovery.

At this stage, a simple portfolio is useful as a control and as the first set of terminal rules. Its limitations are expected. Do not mix architectural debugging with external solver reconstruction before the ownership and rollback rules are reliable.

14.2 Gate 2: witnesses, induction, and exact arithmetic certificates

Port the small certificate formats and replay lemmas. Begin with positive numeric integer witnesses, polynomial accumulator invariants, and weak ordered-polynomial inequalities. Then add parameterized witness guards and dependency-closed motive generalization. The gate is passed by kernel-checked examples, negative mutation tests, and comparisons against the same leaf tactics without synthesis.

The generated scripts in this archive are starting points for this gate, not evidence that it has already passed. A Lean runtime was absent during their generation. The supplied compilation harness records that fact and can run them in an existing compatible Mathlib project.

14.3 Gate 3: demand indexing and persistent cooperation

Implement the conclusion index, typed missing-term proposals, obligation tables, event subscriptions, and proof-scoped fact sharing. Add held-out distractor libraries and recursive theorem dependencies. Then introduce the version-specific grind adapter and compare rebuilding a leaf state against persistent reuse.

The gate requires not just a solve-rate improvement but stable replay after branch pruning, no theorem or local-variable leakage, and no severe regression from index construction. Every new internal hook should have a public-wrapper equivalent for differential testing where practical.

14.4 Gate 4: SAT and external reconstruction

Start with a small, clearly specified Boolean/theory fragment and a replayable coarse loop. Add strict and native profiles as distinct configurations. Reuse Duper and existing SMT adapters before expanding proof-format coverage. Only implement incremental theory propagation after the measured cost of repeated theory checks justifies it.

The gate is passed when every accepted clause has a Lean proof, corrupted traces fail, unsupported proof steps return UNKNOWN, and reported runtime includes preprocessing, external search, reconstruction, and kernel checking. External “unsat” without admissible replay is recorded as a failed proof attempt.

14.5 What not to implement first

Do not begin with unrestricted theorem retrieval, arbitrary higher-order unification, universal e-graph saturation, full quantifier elimination, a custom neural search model, and all SMT theories simultaneously. Each is a separate source of search explosion or reconstruction complexity. A tactic with a few well-integrated, sharply specified components is more likely to be useful than a broad collection of unreliable adapters.

15 Executed experiments and their interpretation

15.1 Environment, counting, and reproducibility

The executed implementation is Python, not Lean. The recorded environment is Python 3.13.5 on x86-64 Linux, with SciPy 1.17.0, NumPy 2.3.5, and SymPy 1.14.0. The pseudorandom seed is 20260914. The test runner uses exact Fraction arithmetic for certificate validation and sets single-threaded BLAS/OpenMP environment variables in the documented reproduction

command.

The aggregate results are generated directly from `results/experiments.json`:

Component	Cases	Accepted checks	Mutations rejected
Nonnegative-polynomial certificates	33	33	36
Integer lattice certificates	689	689	240
Accumulator induction identities	23	23	46
Ground-Horn search cases	88	146	8
Total	833	891	330

A case and a *certificate check* are not the same unit. The Horn experiments include 80 comparisons between two algorithms; when both return a derivation, both derivations are checked. Conversely, a comparison where neither derives the target has no accepted derivation. The counts exclude repeated timing runs and a few additional archived demonstration checks. They are not 891 statistically independent benchmark problems.

There are also 150 polynomial cross-checks against SymPy and 897 comparisons between recursive accumulator execution and the synthesized formula. The latter are secondary tests: the certificate's acceptance condition remains the universal base and step identities. Twenty-six representative certificates are archived, and a separate standard-library-only replay pass accepts all 26.

15.2 Nonnegative-polynomial certificates

Nine hand-selected examples exercise quadratic and quartic squares, a three-premise product, a four-variable Lagrange identity, interval products, equality-ideal terms, and rational weights. Twenty-four generated examples are random positive rational combinations of squares from a known dictionary. These generated examples are property tests, not held-out evidence that the heuristic generalizes to arbitrary inequalities.

Polynomial problem	Candidates	Terms	Median ms
quadratic_dispersion	16	3	2.185
quartic_dispersion	100	3	6.756
positive_triple_product	68	1	3.605
lagrange_identity	225	1	13.146
box_quadratic	12	1	1.756
box_cubic	28	2	2.473
inverse_pair_bound	10	2	1.975
equality_transport	12	2	2.335
weighted_quartic	36	2	2.941

The timings are medians of three calls to discovery, which itself includes exact validation before returning a certificate. They exclude the extra independent validation call performed by the test runner. Import and warm-up effects can influence the first call; three repetitions are not a rigorous performance study. The numbers are useful for reproducing the scale of the prototype, not for comparing it to compiled Lean tactics.

Five expected misses are recorded. Removing premise products prevents the dictionary from proving $x(1 - x) \geq 0$ on $[0, 1]$. Removing binomial squares prevents it from proving $(x - y)^2 \geq 0$ in its expanded representation. The unequal-coefficient square $(2x - 3y)^2$ is outside this

dictionary's quadratic cone. The false claim $x^2 - 1 \geq 0$ is not accepted. Finally, the polynomial

$$x^4y^2 + x^2y^4 + 1 - 3x^2y^2$$

is not certified by the tested dictionary. Its nonnegativity follows from AM–GM applied to its three nonnegative leading terms, but the observed miss does not establish any general impossibility result about other certificate languages.

Corruptions include changing the target, negating a positive coefficient, and replacing premise or equality indices with invalid references. All 36 attempted mutations in this component are rejected. This tests several important failure paths, but does not formally verify the checker implementation.

15.3 Integer-lattice extraction

The one-row grid exhausts $a, b \in \{-3, \dots, 3\}$ and right-hand sides in $\{-5, \dots, 5\}$: 539 systems. Their feasibility is compared independently with the elementary gcd criterion. Another 120 systems are generated from a hidden integer witness, using up to five equations and six unknowns. For each returned affine family, four additional choices of free parameters are substituted into the original equations.

Thirty more systems become inconsistent only after a consistent prefix, for example two proportional left-hand sides with incompatible right-hand sides. Across the 689 principal cases, the checker accepts 537 feasible results and 152 certified infeasibility results. All 240 witness/residual mutations in the randomized feasible tests are rejected.

This tests more than bounded witness search: the checker validates the unimodular explanation and the complete solution-family invariant. It does not yet test parameterized symbolic right-hand sides, inequality residuals, or Lean reconstruction of a negative trace.

15.4 Accumulator invariants

The 23 cases comprise increments n^k for $k = 0, \dots, 8$, two additional hand-selected polynomial increments, and twelve generated rational polynomials. All are solved within the configured degree cap ten. For each, the checker independently expands and verifies the base and step identities.

The test runner also evaluates each recurrence at $n = 0, \dots, 12$ and accumulator values $-3, 0, 4$, giving $23 \cdot 13 \cdot 3 = 897$ execution comparisons. Adding 1 to an invariant corrupts its base case, while adding n corrupts the step relation; all 46 such mutations are rejected. The no-accumulator and insufficient-degree ablations return UNKNOWN.

These results substantiate the restricted coefficient-synthesis algorithm. They do not evaluate automatic discovery of arbitrary recursors, dependent generalization, or library-scale invariant grammars. The prototype receives the accumulator recurrence schema; it is not a Lean source-code analyzer.

15.5 Demand slicing with distractors

The useful Horn chain has 25 rules, including its initial fact. Each distractor chain also has 25 rules and is deliberately placed before the useful chain in the input order. This is a stress microbenchmark designed to expose wasted forward work.

Distractor chains	Indexed rules	Forward firings	Demand firings	Times (ms)
0	25	25	25	0.057 / 0.045
10	275	275	25	0.238 / 0.081
100	2525	2525	25	3.693 / 1.015
1000	25025	25025	25	58.974 / 11.425

The last column reports forward time followed by demand-sliced time. In the largest case, activating a target dependency slice reduces firings from 25,025 to 25. Both methods still receive and index all 25,025 rules. The timing improvement is therefore much smaller than the 1001-fold reduction in firings, and the example must not be reported as a 1001-fold end-to-end speedup.

Eighty random cyclic Horn programs compare the two algorithms' derivability decisions, and every returned derivation is replayed. Eleven of those programs have no derivation for the chosen target under either method; they contribute comparison tests, not proofs of the target. Truncating the scaling examples' proof traces removes the target derivation and is rejected in all eight attempts.

The experiment is not a comparison with Lean's E-matching indexes, existing relevance heuristics, or proof pruning. It demonstrates the standalone slice algorithm on finite ground Horn rules.

15.6 What the artifact does not demonstrate

Delivered and executed	Designed but not executed as a Lean tactic
Exact sparse polynomial arithmetic and independent SymPy cross-checks	Lean expression reification and type-preserving projection
Finite cone-certificate discovery and exact replay	A registered grind satellite for ordered nonlinear goals
Integer-lattice witnesses and negative trace replay	Parameterized witnesses with Lean-proved guards and residual inequalities
Polynomial accumulator-invariant synthesis	Automatic recursor selection and dependent motive generalization
Ground-Horn demand slicing	Full backward retrieval and E-matching cooperation in Lean
Generated Lean/Mathlib replay source	Compilation, kernel checking, axiom audit, and baseline timing of those files
Certificate mutation tests	A hardened external certificate parser or formally verified Python checker

The missing Lean execution is recorded in `results/lean-status.json`; it is not hidden as an assumed success. None of the measured Python timings is a Lean performance result.

16 The benchmark required to establish superiority

16.1 Baselines and two distinct experimental questions

Use the same Lean toolchain, imported modules, theorem database, options, machine limits, and trust profile for every competing configuration. The primary baseline is unmodified grind. Also include tuned grind with a predeclared tuning budget; an ordinary tactic portfolio; Aesop with the same arithmetic leaves; `nlinarith/omega` on their applicable fragments; Duper; and existing SMT integrations where the required dependency combination is available.

The first question is *equal-budget utility*: how many goals does each method solve with the same total CPU, wall-clock, and memory limits? The second is *incremental coverage after baseline failure*: what additional goals can the enhancement solve with an explicitly additional budget? Report them separately. Baseline-first preservation mode is useful evidence for the second question, not a shortcut to winning the first.

Historical egg configurations can be included as a release-pinned research comparison, but its deprecated current status should be visible. A proof-producing strict configuration and a native-trust configuration must appear in different result columns.

16.2 Corpus strata

A useful corpus has several separately reported strata. Existing grind regression tests check preservation. Mathlib-style arithmetic and finite-algebra goals test core cooperation. Recursive programs with newly defined names test induction and helper-lemma discovery without merely retrieving a theorem that already states the answer. Existential problems test witnesses with equality, inequality, divisibility, and type constraints. Boolean-heavy goals test the SAT lane. Dependent types, empty types, casts, and heterogeneous equality test semantic boundaries.

Include mixed problems deliberately requiring more than one component: an accumulator invariant followed by an inequality; a lattice witness constrained by a bound; a backward theorem application requiring a new polynomial certificate; or a Boolean branch whose contradiction needs a typed arithmetic lemma. Compare against the same leaf solvers without the cooperation broker. Otherwise, an apparent improvement may simply reflect adding one stronger standalone solver.

Generated test families must distinguish construction-aware property tests from held-out problems. The current random SOS tests were generated from the solver's own dictionary and belong in the former category. Renaming a theorem while retaining its proof template is not a robust train/test split.

16.3 Avoid theorem leakage

Remove the theorem under test from retrieval, including aliases or stronger equivalent facts that were introduced only as part of the test's reference proof. Preserve ordinary background lemmas consistently across methods. Freeze global attributes and record which definitions may be unfolded.

When comparing an invariant synthesizer with a baseline, do not give the synthesized invariant to the enhanced run for free and then claim that the tactic discovered it. Record the time and search used to obtain it. Likewise, if an external tool provides a premise set or hint, charge its search and reconstruction to the enhanced method.

16.4 Metrics beyond a solved percentage

Report root goals solved, unique additional solves, regressions, timeouts, unsupported-fragment failures, and replay failures. Separate preprocessing, search, proof reconstruction, and kernel checking times. Record peak memory, generated terms/clauses, certificate bytes, and proof DAG size. Report both cold and warm import/cache conditions where they materially affect measurements.

Use multiple deterministic seeds for generated families and repeated runs for wall-clock results.

Pair comparisons by goal. Show family-level breakdowns so that thousands of easy generated arithmetic cases do not overwhelm a small but important induction or dependent-type stratum. Confidence intervals and paired significance tests become useful only after collecting a real comparable corpus; none is inferred from the present microbenchmarks.

16.5 Required ablations

Disable each addition independently: backward retrieval, demand slicing, missing-term construction, witness extraction, motive generalization, invariant templates, cone certificates, alternative forms, SAT cooperation, and persistent solver state. Compare the integrated system with a portfolio containing the same leaf procedures and the same total budget.

For induction, compare no generalization, changed-parameter generalization, and dependency-closed generalization. For nonlinear search, compare monomial squares, binomial squares, weighted quadratic seeds, premise-product depth, and equality-ideal terms. For scheduling, compare fixed priorities, fair widening, and any learned policy. This isolates which mechanisms actually contribute rather than attributing every solve to the architecture as a whole.

16.6 Soundness and replay regression suite

The suite should deliberately exercise branch-local fact leakage, escaped existential variables, stale e-class identifiers, dependent transport, empty-domain assumptions, characteristic mismatches, division by zero, natural subtraction, signed/unsigned confusion, wrong bit widths, negative “SOS” weights, and unapproved axioms. Mutate certificates and test parser resource limits.

For successful examples, replay in a fresh process with discovery disabled. Verify the theorem’s transitive axiom set, not merely the absence of the word `sorry` in its source. Test rollback by running an intentionally failing branch before the successful one and checking that the resulting proof and context are unchanged.

17 Assessment and recommended direction

The strongest engineering opportunity is not to rebuild `grind`’s existing arithmetic or equality machinery. It is to give that machinery a disciplined source of new proof structure and useful intermediate facts. Backward demands discover missing premises; witness synthesis produces terms with checked side conditions; induction generalization makes recursive hypotheses usable; exact certificates turn untrusted algebraic search into small proof obligations; and a separate Boolean lane addresses a different search regime.

The prototype work supplies concrete evidence for four pieces of that plan. The lattice procedure gives complete integer-equation families and replayable obstructions. The accumulator procedure covers polynomial increments within its degree limit through exact coefficient solving. The cone search finds useful ordered-polynomial certificates and rejects invalid ones. The Horn experiment demonstrates a sound relevance-slicing mechanism with explicit indexing costs.

What remains is substantial but well delineated: implement the Lean proof adapters, build the owned obligation DAG, connect it to `grind` without leaking scope or metavariable state, and run the matched-budget benchmark. The proposed tactic is not yet an empirically superior replacement. It is a technically specified route to broader automation, supported by executable component algorithms rather than an untested list of solver names.

A Artifact map and reproduction

Path	Contents
article/forge.tex	Complete article source, with bibliography in the same file.
article/forge.pdf	Rendered and inspected PDF.
article/table-*.tex	Tables generated from the recorded JSON.
prototype/polynomial.py	Sparse exact rational polynomials and linear solving.
prototype/sos.py	Finite cone discovery and exact certificate checker.
prototype/lattice.py	Unimodular integer witness algorithm and independent replay.
prototype/induction.py	Accumulator-template synthesis and universal identity checks.
prototype/horn.py	Ground-Horn forward evaluation and demand slicing.
prototype/run_experiments.py	Seeded experiments, expected misses, and mutations.
prototype/verify_artifacts.py	Standard-library-only replay of saved certificates.
prototype/emit_lean.py	Generates ordinary Mathlib proof-source candidates from verified certificates.
prototype/make_tables.py	Regenerates the data-derived tables.
results/experiments.json	Full recorded experiment results and environment.
results/certificates/	Twenty-six representative JSON certificates.
results/replay.json	Outcome of the separate archived-certificate replay.
results/lean-status.json	Explicit record that Lean compilation was not run.
lean/GeneratedSOS.lean	Nine emitted nonnegative-polynomial proof scripts; uncompiled here.
lean/GeneratedInduction.lean	Eleven emitted accumulator definitions and invariant proof scripts; uncompiled here.
lean/WitnessExamples.lean	Three illustrative ordinary witness/obstruction proofs; uncompiled here.
lean/run_checks.py	Compilation harness for an existing compatible Mathlib project.
sources.json	Source URLs, observation date, and available inspected blob identifiers.

From the archive root, replaying the saved certificates needs no third-party Python package:

```
python prototype/verify_artifacts.py
```

To repeat discovery and the randomized tests, install the versions recorded in `prototype/requirements.txt`, then run:

```
# POSIX shell; do not run with Python's -O flag.
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 \
  python prototype/run_experiments.py
python prototype/verify_artifacts.py
python prototype/emit_lean.py
python prototype/make_tables.py
```

The test runner uses assertions and writes a new results JSON. Timings are expected to change. Preserve the supplied results when comparing another machine's run. The README also gives PowerShell commands.

The supplied Lean files are ordinary Mathlib proof candidates, not an installed `forge` tactic. To compile them in an already compatible project:

```
python lean/run_checks.py --project /path/to/mathlib-project \
  --output local-lean-results.json
```

The harness neither installs packages nor modifies the target project's configuration. It reports missing executables and compilation failures explicitly. Kernel compilation must succeed before these files are described as verified Lean proofs.

Build the article from its own directory with XeLaTeX twice:

```
cd article
xelatex -interaction=nonstopmode -halt-on-error forge.tex
xelatex -interaction=nonstopmode -halt-on-error forge.tex
```

The source uses standard TeX packages, an open sans-serif text font, a monospaced font, and Latin Modern Math. No font binaries are distributed in the archive.

B Proposed frontend and diagnostic contract

A production frontend might expose the following syntax. It is a design example, not syntax recognized by the supplied files:

```
forge
forge?                -- emit an explicit replay plan
forge only [h1, h2, lemmaA]  -- constrained theorem input
forge (profile := strict)
forge (strategy := preserveGrind)
forge (induction := true) (coneDegree := 4)
forge_trace           -- inspect unfinished obligations
```

An informative failure should identify the obstructing obligation and the relevant limit rather than print an undifferentiated state dump. Examples include “candidate witness needs proof that 2 divides n ,” “induction hypothesis cannot match a changing accumulator,” “cone certificate needs an unproved denominator sign,” and “external unsat result could not be reconstructed in the selected trust profile.”

A successful replay record should contain the selected theorem premises, witness terms, generalized variables, recursors, certificate identities, dependency fingerprints, and final axiom profile. Optional search traces can explain rejected candidates, but should not be required to replay the proof.

A stable machine-readable result distinguishes successful proof, exhausted budget, unsupported fragment, failed reconstruction, and invalid certificate. A verified concrete counterexample can have its own result form; an unchecked model or failed proof attempt cannot use that status.

C Certificate boundaries at a glance

Certificate	Search may be untrusted	Acceptance condition
Cone identity	Numeric LP, dictionary heuristics, square proposals	Exact identity, nonnegative weights, valid proved premises, correct ordered-field interpretation
Integer lattice	Gcd/normal-form search, basis choice	Exact unimodular operations and residuals; direct witness substitution for positive existential use
Accumulator invariant	Template selection, coefficient synthesis	Universal base and step identities, followed by the real induction rule
Horn derivation	Demand slicing, rule order	Every step is an input rule whose premises are already derived
Equational path	E-graph saturation, term extraction	Typed theorem instances and congruence/transport explanations in the proper scope
SAT/theory proof	SAT search, theory heuristics, core minimization	Valid propositional trace and Lean proof of every theory clause and abstraction bridge
SMT hint	External solver, learned ranking	A reconstructed Lean proof of the hinted fact before publication

The first four certificate families have executable Python implementations in this archive. The remaining rows specify the intended Lean system's contracts. Keeping this distinction explicit is essential both for evaluating the present work and for developing the tactic without expanding its trusted base accidentally.

References

- [1] Lean project. *The Lean Language Reference*, current version observed as 4.34.0 on September 14, 2026. <https://lean-lang.org/doc/reference/latest/>
- [2] Lean project. *Lean v4.34.0 release*, published September 14, 2026. Release metadata inspected through the GitHub API. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>
- [3] Lean project. *The grind tactic*. Reference manual, observed September 14, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>
- [4] Lean project. *Algebraic Solver (Commutative Rings & Fields)*. Reference manual. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings___-Fields_RPAR_/
- [5] Lean project. `src/Lean/Meta/Tactic/Grind.lean`, tag v4.34.0. Inspected blob: db642e7864020148fab27822bf83c445de83d02d. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind.lean>
- [6] Lean project. `src/Lean/Meta/Tactic/Grind/Action.lean`, tag v4.34.0. Inspected blob: ad212d7c8ef8277b4be3650d43738ee98a32cbf5. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Action.lean>
- [7] Lean project. `src/Lean/Meta/Tactic/Grind/Finish.lean`, tag v4.34.0. Inspected blob: 6bae01e1c2c4772d0cabe99ff74437492ccebfc5. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Finish.lean>
- [8] Lean project. *Tactic Reference*. Sections on induction, integer arithmetic, SAT integration, native evaluation, and proof-producing call-by-value evaluation. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>
- [9] Lean community. *Aesop* repository and README. Observed README blob: f75ec1f8bebcf6d857fc3954cf91cf99f323a758. <https://github.com/leanprover-community/aesop>
- [10] Evan Wang, Simon Chess, Sophie Szeto, and Theodore Meek. *Learned Interventions in Lean 4 grind*. arXiv:2607.22972v2, July 2026. <https://arxiv.org/abs/2607.22972>
- [11] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. Documentation and implementation overview, including nonlinear preprocessing. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html
- [12] Mathlib contributors. *Mathlib.Tactic.Positivity.Core*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Core.html
- [13] SciPy developers. *scipy.optimize.linprog*. API documentation. The experiments used SciPy 1.17.0, not an assumed latest installed version. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>
- [14] SymPy developers. *Polynomial-domain matrix documentation*. The experiments used SymPy 1.14.0 for independent symbolic expansion checks. <https://docs.sympy.org/latest/modules/polys/domainmatrix.html>
- [15] Marcus Rossel and contributors. *lean-egg* README. Deprecation notice observed September 14, 2026; blob 0dbfa063b5a77c6e892eaa265f068437980812c1. <https://github.com/marcusrossel/lean-egg>
- [16] Marcus Rossel, Rudi Schneider, Thomas Koehler, Michel Steuwer, and Andrés Goens. *Towards Pen-and-Paper-Style Equational Reasoning in Interactive Theorem Provers by Equality Saturation*. Proceedings of the ACM on Programming Languages, 2026. DOI: 10.1145/3776667. <https://doi.org/10.1145/3776667>

- [17] The egg contributors. *egg: e-graphs good*. Project and library documentation. <https://egraphs-good.github.io/> <https://github.com/egraphs-good/egg>
- [18] Joshua Clune, Yicheng Qian, Alex Bentkamp, and contributors. *Duper* repository and README. Observed README blob: 1859f7fa914fbd8151b274fd34e46f3a09ef1b2d. <https://github.com/leanprover-community/duper>
- [19] Lean community. *lean-auto*: interface to automated theorem provers. <https://github.com/leanprover-community/lean-auto>
- [20] Joshua Clune, Haniel Barbosa, and Jeremy Avigad. *Hint-Based SMT Proof Reconstruction*. arXiv:2601.14495, January 2026. <https://arxiv.org/abs/2601.14495> <https://github.com/JOSHCLUNE/QuerySMT>
- [21] Abdalrhman Mohamed, Tomaz Mascarenhas, Harun Khan, Haniel Barbosa, Andrew Reynolds, Yicheng Qian, Cesare Tinelli, and Clark Barrett. *Lean-SMT: An SMT tactic for discharging proof goals in Lean*. arXiv:2505.15796, May 2025. <https://arxiv.org/abs/2505.15796>
- [22] cvc5 developers. *Proof Production*. Current documentation describing the Cooperating Proof Calculus and external proof formats. <https://cvc5.github.io/docs/latest/proofs/proofs.html>